

dsupdt

Periodic Dataset Updates for the GDEX Servers

Download → Build → Archive, scheduled and automated

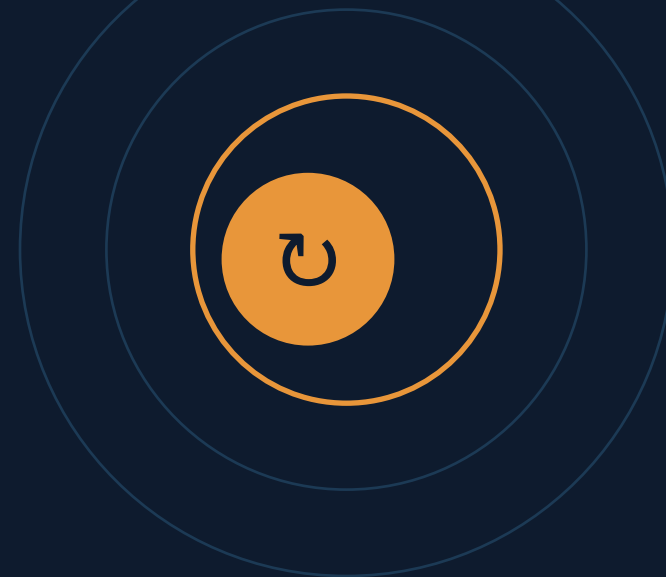
Developer Configuration

General Usage

Updated 2026-07

Zaihua Ji zji@ucar.edu

Companion utility to `dsarch` • orchestrated by the `dscheck` daemon



What is dsupdt?

dsupdt performs **periodic dataset updates** — it retrieves data from remote servers, assembles archive-ready local files, and hands them to **dsarch** for archiving onto the GDEX Servers.

Configured per dataset in GDEXDB

All operational datasets run under **dsupdt / dsarch**; configuration spans three record types.

Edit records from the CLI (

-GA / -SA) or the web Config Editor (next slide).

Only the owning specialist may run an update record (override with

-MD).

Input files must be named

dNNNNNN . * to guard against the wrong dataset.



Configure

update controls, local & remote file records



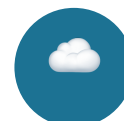
Download

copy / fetch server files to a working area



Validate & Build

run specialist routines, tar & compress



Archive

hand local files to dsarch → GDEX



Clean

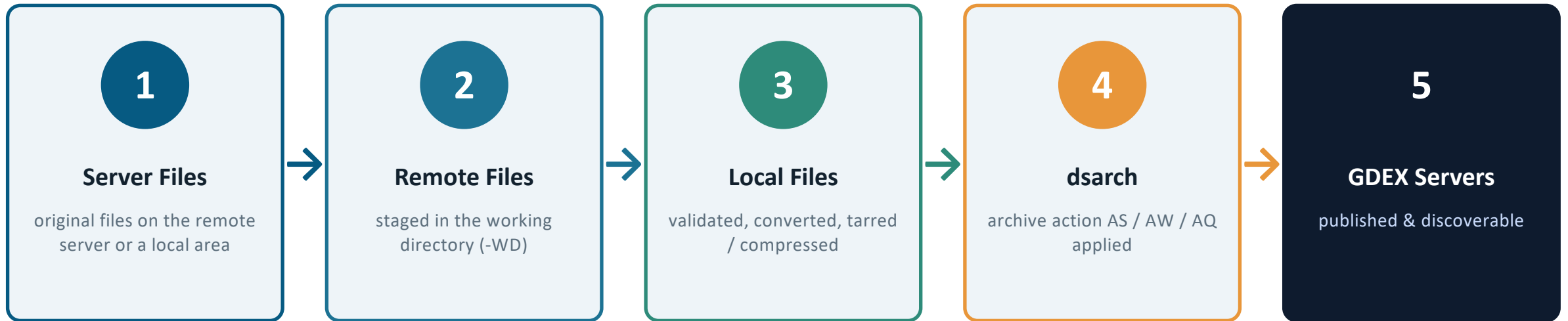
remove temporary files after success



Check

test remote-file availability

The Five-Stage Update Pipeline



One command runs the whole chain. `-UF` (`-UpdateFile`) performs all stages end-to-end. The individual steps `-DR`, `-BL`, `-PB`, `-AF`, `-CF` let you drive each stage separately.

Three Record Types in GDEXDB

Update Control

dcupdt -SC / -GC

- Schedules when updates run
- Frequency, offset, retry, valid interval
- Email & error controls
- Launched by the dscheck daemon

Local File

dlupdt -SL / -GL

- Minimum config for an update
- Local file name & dsarch action
- Download / build / clean commands
- Tracks next end date/hour

Remote File

drupdt -SR / -GR

- Only when remote ≠ local name
- Many remotes → one local file
- Server file & download order
- Time interval for sub-periods

Get → edit → Set: -GA (-GetAll) dumps all three sections; -SA (-SetAll) applies an edited file back to GDEXDB.

Anatomy of a Command

```
dsupdt [-DS dNNNNNN] <Action> [Mode...] [Info...]
```

Action

One per run

Selects the task. Takes no value. E.g. -UF, -GC, -SL, -AF. Some actions imply others.

Mode

Zero or more

Modifies behavior; takes no value. E.g. -MU, -FU, -RA, -NE, -GZ.

Info

Carries data

Supplies values: dataset, indices, dates, file names, commands. E.g. -CI, -ED, -LF.

Notation & filters (A|B) short|long form • option names are case-insensitive, values are not • Get-style filters accept ! < > <> • the first -DS may omit its option name.

Quick Start

Show a single option's help

```
dsupdt -h -UF
```

Get update control records

```
dsupdt d540000 -GC
```

Get local file records

```
dsupdt d540000 -GL
```

Run download–build–archive

```
dsupdt d540000 -UF
```

Process all elapsed periods

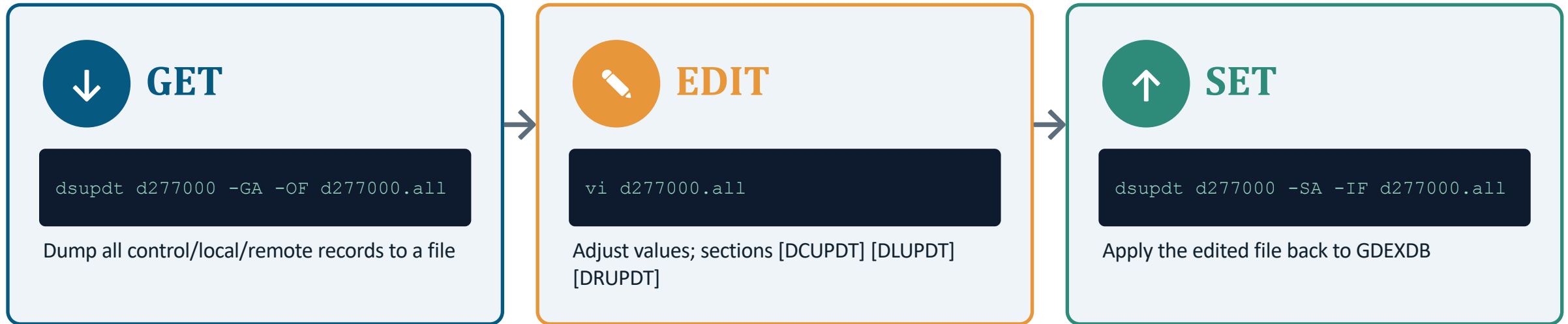
```
dsupdt d540000 -UF -MU -IE
```

Check remote-file availability

```
dsupdt d540000 -CU
```

Tip: run **dsupdt** with no arguments to page the full guide through **more**. Omit **-DS** on Get actions to list records across all of your datasets.

Configuration Workflow



Input file (-IF) format

...

Dataset<=>d277000

SC<!>

col1<:>col2<:>

dNNNNNN.*

comment line

single-value assignment (-ES sets '<=>')

Action / Mode marker (-AO sets '<!>')

tabular header row (-DV sets '<:>')

file name must start with the dataset number

Web Config Editor

The same three record types can be edited in a browser — no **-GA** / **-SA** Get/Edit/Set round-trip. Open the **RDAMS PostgreSQL Configuration Editor** and expand the **DSUPDT** menu.

```
gdex.ucar.edu/rda_pg_config > DSUPDT > Update Control | Local File | Remote File
```

Update Control

```
dsupdt_control.php
```

dcupdt • **-SC** / **-GC**

frequency, next-due, actions, e-mail & error controls

Local File

```
dsupdt_localfile.php
```

dlupdt • **-SL** / **-GL**

file names, patterns, build / convert commands, archive action

Remote File

```
dsupdt_remotefile.php
```

drupdt • **-SR** / **-GR**

remote & server names, download command & order



Split-pane editor: pick a dataset & record on top, edit fields below. A record is **locked** while open; add, edit, save, or delete, then release. Writes go straight to GDEXDB — identical to the CLI.

Update Control Record (-SC)

Schedules automatic actions launched by **dscheck**. New record: index 0 + **-NC**. Existing index is modified in place.

Opt	Long name	Purpose
-AN	ActionName	dsupdt task: UF, DR, BL, PB, CL, CU
-FQ	Frequency	how often, e.g. 1W, 1M, 6H, 1M/3
-CO	ControlOffset	delay after period end, e.g. 2D10H
-CT	ControlTime	next scheduled run YYYY-MM-DD HH:NN:SS
-RI	RetryInterval	wait before retry after a failure
-VI	ValidInterval	how long remote files stay valid
-PI	ParentIndex	run only after parent control completes
-UC	UpdateControl	preset modes: B C E F G M N O Y Z
-MC	EMailControl	A / S / E / B / N report level
-EC	ErrorControl	I ignore / Q quit / N normal
-KF	KeepFile	S / R / B / N which files to keep
-HN	HostName	restrict/allow processing hosts

Local File Record (-SL)

The **minimum** configuration to update a file. Link to a control via **-CI** so its scheduled action runs automatically. New: **index 0 + -NL**.

Opt	Long name	Purpose
-LF	LocalFile	name, may embed temporal patterns
-AN	ActionName	dsarch action: AS, AW, AQ
-OP	Options	options string passed to dsarch
-DC	DownloadCommand	fetch/copy/generate the file
-FQ	Frequency	data cadence, e.g. 1M, 1W, 6H
-ED	EndDate	data end date of next update
-EH	EndHour	end hour (sub-daily cadence)

Opt	Long name	Purpose
-DI	DueInterval	end → ready lead time
-VI	ValidInterval	re-check / hold-open window
-AT	AgeTime	min remote-file age to fetch
-WD	WorkDir	staging dir (root \$UPDTWKP)
-PR	ProcessRemote	user validate/convert cmd
-BC	BuildCommand	user build cmd (vs tar)
-CL	CleanCommand	remove temp files after

Patterns in -LF / -RF / -DC / -DE (default delimiters < >) are substituted at update time, e.g. **file.<YYYYMM>.ext** → **file.200710.ext**.

Remote File Record (-SR)

Add a remote record only in these three cases:



Different name

remote file name differs from the local file name



Many → one

several remote files are tarred into one local file



Multiple servers

one file available from several servers (-DO order)

Opt	Long name	Purpose
-LI	LocalIndex	local record this remote belongs to (required)
-RF	RemoteFile	remote name(s); join many with ':' ; '!' = shell cmd
-SF	ServerFile	name on server if different from remote name
-DC	DownloadCommand	overrides the local record's command
-DO	DownloadOrder	try lowest index first, then ascending
-TI	TimeInterval	one name per sub-period, e.g. 6H, 5D
-BT	BeginTime	window start (with -TI)
-ET	EndTime	window end (with -TI)

The Update Action Family

-UF

Update File

end-to-end: download → build → archive → clean

-DR

Download Remote

fetch/copy/generate remote files

-BL

Build Local

validate, convert, tar/compress

-PB

Process Both

download + build in one step

-AF

Archive File

hand local files to dsarch

-CF

Clean File

remove temporary working files

-CU

Check Update

test remote-file availability

-UL

Unlock

clear locks from aborted runs

-UF bundles the four steps. Stepping manually is useful for debugging or re-running a single stage. All actions accept **-CI** / **-LI** / **-LF** / **-XO** to target records, and **-ED** / **-EH** / **-CD** / **-CH** to override dates at run time.

-UF Deep Dive: One Command, Full Cycle

For each due local file record

1. Download / copy the server file
2. Validate & convert (-PR) into a remote file
3. Build the local file (tar / compress or -BC)
4. Archive via dsarch using the record's action
5. Clean temporary files (-CL)
6. Advance data end date/hour & next-due times

Mode/Info	Effect
-MU	process every elapsed period (else newest only)
-FU	force at least one update even if not due
-CP	allow the current, not-yet-complete period
-RA	re-archive (pass -RA to dsarch)
-RD	re-download even if the file exists locally
-MO	only periods not yet archived
-IE	skip failures & continue (with -MU)
-QE	stop the dataset on first error
-PL	fork N child processes for parallel records
-CC	add carbon-copy email recipients

```
dsupdt d337000 -UF -MU -IE -CC schuster
```

Archiving & Content Metadata

Once a local file is built, **dsupdt** hands it to **dsarch** to archive onto GDEX, then optionally gathers content metadata so the new data is discoverable.



Archive to GDEX — dsarch

dsupdt calls dsarch with the record's action (AS / AW / AQ). -OP appends extra dsarch options; -RA forces re-archiving. This is the step that publishes each file.



Gather content metadata — gatherxml (-GX)

Optional. When -GX is set in the local record's Options, dsupdt runs gatherxml on the just-archived file to index its variables and spatial / temporal coverage.



Refresh the search index — scm

After archiving, dsupdt runs the search-metadata command (scm -d DS -r w) to summarize / refresh the dataset's web tables so the new content is searchable.

Metadata steps are non-fatal: a gatherxml or scm failure is reported in the e-mail but never blocks or retries the archive.

Temporal & Generic Patterns

Placeholders in file names, download commands, and descriptions are substituted at update time. Default delimiters < > (change with -PD). By default patterns use the period's **end** date/hour.

Pattern	Basis	Meaning
<YYYYMM>	end date	4-digit year + 2-digit month → 200710
<YYYY/MM/DD>	end date	path-style date components
<CYYYY.MM.DDC>	current	wrap in C ... C to use the current date/hour
<BYYYY.MM.DDB>	begin	wrap in B ... B to use the period's begin time
<M*M>	fraction	per-fraction id: C=A,B,C c=a,b,c N=1,2,3
<HH>	end hour	2-digit hour for sub-daily cadence
<P0> <P1>	runtime	generic values supplied via -GP

Example

```
-LF uv.<YYYYMM>.bln
```

```
end date 2007-10-31
```

↓ **becomes**

```
uv.200710.bln
```

Dynamic work dir: when a server path carries a date the remote name lacks, embed that date in -WD so different dates don't overwrite each other.

Custom Commands with '!'

Some fields are shell commands by design (**-DC** / **-BC** / **-CL**). Any value field can also become a command by prefixing it with **'!'** — dsupdt runs it and uses the stdout as the value. Temporal patterns are substituted first.

-DC**Download Command**

How each remote file is fetched from the server (wget, cp, mftp, ...).

-BC**Build Command**

External utility that assembles / converts the local file after download.

-CL**Clean Command**

Post-archive cleanup command for temporary or intermediate files.

-RF '!...'**Remote File**

Name resolved at run time — e.g. pick the newest matching server file.

-LF '!...'**Local File**

Compute the local file name from a script instead of a fixed pattern.

-DE '!...'**Description / note**

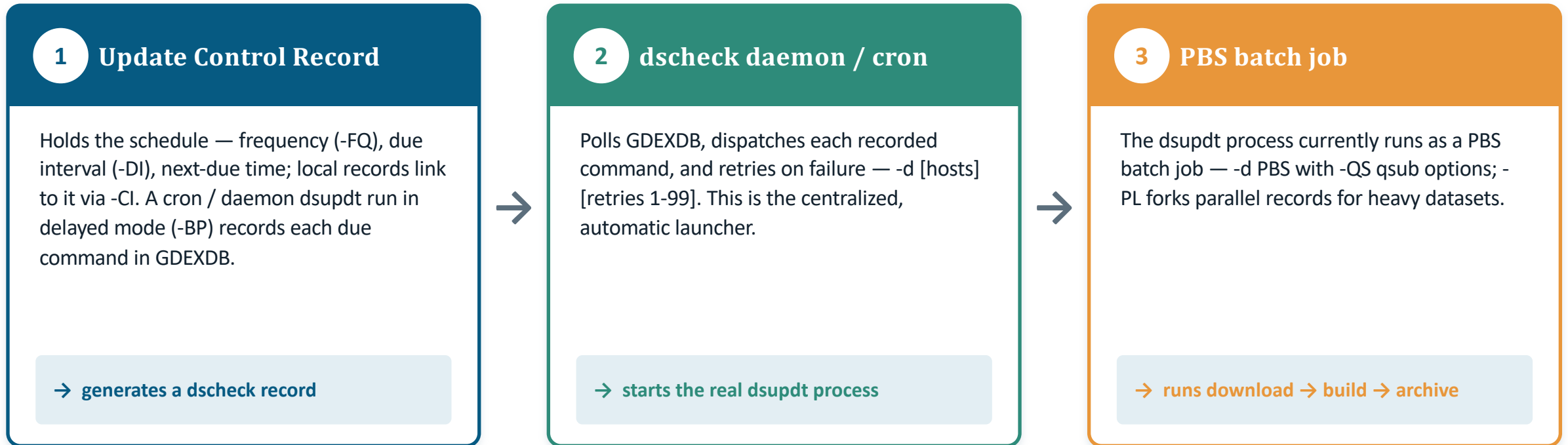
Generate a data note dynamically from command output.

Example — pick the newest matching file at update time

```
dsupdt d540000 SR -LI 1 -RF '!ls -t /glade/incoming/<YYYYMM>*.grib | head -1'
```


Scheduling & Execution

In production the whole chain is automated — currently under **PBS batch control**:



Direct / manual: run `dsupdt -UF` yourself — or from a plain cron job on a specific host — for one-off runs, re-archiving, and debugging. No control record or dscheck needed.

Valid Interval & Multi-Period Updates

-FQ Frequency

the size of one update period (1D, 6H, 1M, ...)

-MU MultipleUpdate

process every elapsed period, not just the newest

-VI ValidInterval

look back this far to re-check / re-archive late data

-DI DueInterval

lead time from period end until data is expected

-IE / **-QE** / **-EC** govern how errors interact with multi-period runs: skip & continue, quit, or normal.

Re-check window (VI = 4D, FQ = 1D)



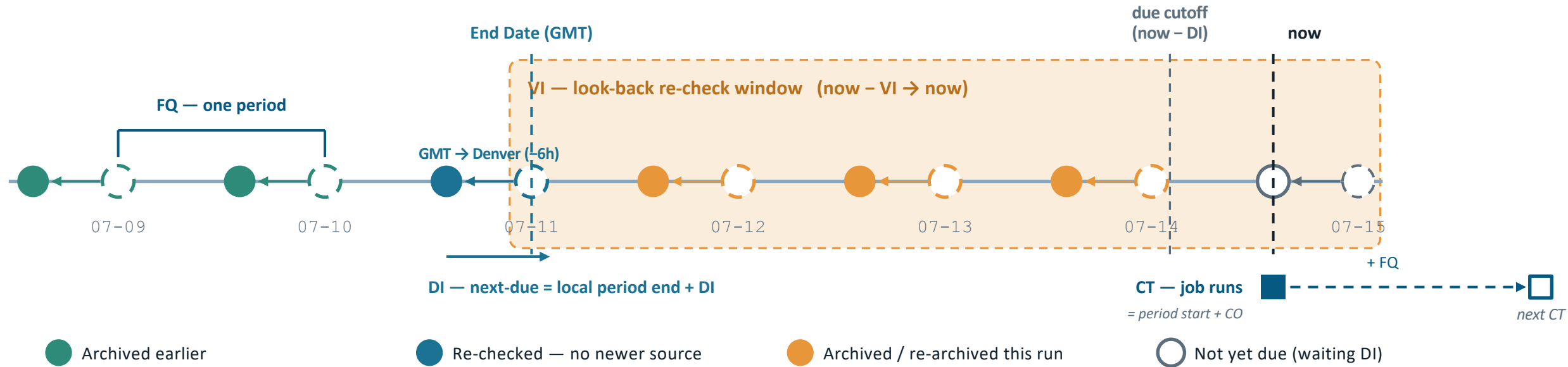
Each period is re-checked. Already-archived periods report “no newer file”; a newer source file triggers **re-archive**. VI = 0 → single period, no newer-file check.

Partial updates

With **-MR Y**, a many-to-one build tolerates missing remotes. If **-VI** is also set, the update is held open until the files arrive or the window expires.

How the Temporal Intervals Fit Together

Data advances one **Frequency (FQ)** per period. A single **-MU** run walks every elapsed period inside the **Valid Interval (VI)** look-back window, while the **Due Interval (DI)** sets when each period becomes due.



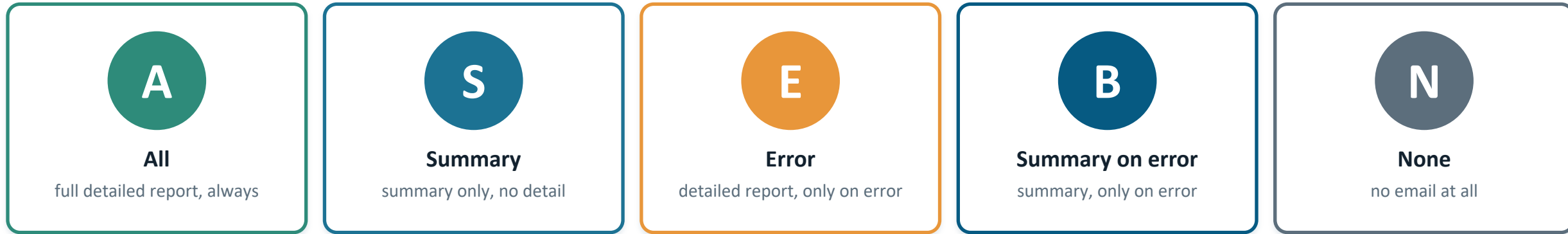
ControlTime — when the job itself runs

-CT sets the run time (YYYY-MM-DD HH:NN:SS). Each success steps it by one **-FQ** (Next CT = CT + FQ); **-CO** only offsets the run from the period start (daily → day start + CO), so it cancels in the step. Failure retries after **-RI**. e.g. `dsupdt d609000 -SU -CT "2026-07-11 09:00:00" -FQ 1D -CO 9H`

Time zones: the data end date/hour (End Date, **-ED** / **-EH**) is server time (GMT), while **-CT** is local time the job runs (Denver, MDT = GMT-6h in summer) — don't mix the two.

Each run re-checks every period in the VI window: already-archived periods report “no newer file”, newly available periods are archived, and periods still within DI of now are left for the next run. $VI = 0 \rightarrow$ only the most recent period.

Email Reporting (-MC / -EMailControl)



Report sections

Error	real errors — survives in every mode; raw failures are prefixed with the failing command / dataset
Summary	headline count + rolled-up “N files ARCHIVED” range lines
Detail	per-file lines; consecutive no-op re-checks collapse into one range line

Related: `-EE` (=E), `-SE` (=S), `-NE` (=N), `-CC` adds carbon-copy recipients.

Key Mode Options at a Glance

-MU process all elapsed periods	-FU force one update even if not due	-CP allow current, not-yet-due period
-MO only not-yet-archived periods	-RA force re-archive via dsarch	-RD re-download existing remote file
-CN re-download if changed on server	-RE reset end date/hour from file time	-IE skip errors, continue (with -MU)
-QE quit dataset on first error	-GZ use GMT as controlling time	-NY skip Feb 29 in leap years
-KR keep remote file (copy, not move)	-KS keep server file (copy, not move)	-UB use period begin time for patterns
-UT force end/next-due time advance	-BG background; suppress screen output	-NE suppress notification email

Most modes can be preset in the control record via -UC (single letters) and -KF / -MC / -EC.

Environment & Prerequisites



Under dsarch/dsupdt

All operational datasets run under dsupdt/dsarch control; dsupdt hands its local files to dsarch for archiving onto GDEX.



\$UPDTWKP

Root for working dirs. If set to /lustre/gdex/work, then \$UPDTWKP/zji/icoads resolves under it; if unset, -WD is used as given.



Ownership

Only the owning specialist may run a record. Use -MD to override, or -LN to run on behalf of another (auto-adds -MD to dsarch).



dNNNNNN.*

Input files must start with the dataset number and match -DS — a guard against operating on the wrong dataset.



-PL parallelism

Fork N child processes, each handling one record, for datasets with many independent, time-consuming updates.



-DB / logs

Debug logging via -DB level [path] [file]; default log path \$DSSHOME/dssdb/log, file pgdss.dbg.

Troubleshooting & Recovery

Stuck lock

A crashed run may leave PID/host on a record, blocking reprocessing on another host.

```
dsupdt -UL -CI 57  
dsupdt -UL -LI 33
```

Is data ready?

Test remote-file availability before committing to an update.

```
dsupdt d540000 -CU -CA
```

Re-run a period

Substitute a different date to re-archive or recover a deleted action.

```
dsupdt d540000 -UF -CD 2026-07-01 -RA
```

Retry on failure

Control records retry automatically after -RI; -EC sets error behavior.

```
dsupdt d540000 -SC -CI 5 -RI 12H -EC N
```

Recap

From Config to Archive

- 1 **Configure** three record types: control (dcupdt), local (dlupdt), remote (drupdt)
- 2 **Run** -UF drives download → build → archive → clean in one command
- 3 **Automate** link to a control record; dscheck dispatches, retries, and reports
- 4 **Tune** patterns, valid interval, parallelism, and email controls per record

Learn more `dsupdt -h <OPT>` • full guide: dsupdt.usg • docs: gdex-docs-dsupdt.readthedocs.io • repo: github.com/NCAR/rda-python-dsupdt



Any Questions?

Thank you

`dsupdt -h <OPT>` • dsupdt.usg • github.com/NCAR/rda-python-dsupdt